PRACTICAL SESSION 8

Task 1: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
  UW PICO 5.09                          File: memory_allocation.c

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int i;
    int *ptr;

    /* malloc(): allocates memory */
    ptr = malloc(3 * sizeof(int));

    if (ptr == NULL) {
        printf("malloc failed\n");
        return 1;
    }

    ptr[0] = 10;
    ptr[1] = 20;
    ptr[2] = 30;

    printf("Values using malloc: ");
    for (i = 0; i < 3; i++) {
        printf("%d ", ptr[i]);
    }
    printf("\n");

    /* realloc(): changes allocated memory size */
    int *temp = realloc(ptr, 5 * sizeof(int));

    if (temp == NULL) {
        free(ptr);
        printf("realloc failed\n");
        return 1;
    }

    ptr = temp;
    ptr[3] = 40;
    ptr[4] = 50;

    printf("Values after realloc: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", ptr[i]);
    }
```

```
  UW PICO 5.09                          File: memory_allocation.c

    free(ptr);
    ptr = NULL;

    /* calloc(): allocates zero-initialized memory */
    int *zero_ptr = calloc(5, sizeof(int));

    if (zero_ptr == NULL) {
        printf("calloc failed\n");
        return 1;
    }

    printf("Values using calloc: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", zero_ptr[i]);
    }
    printf("\n");

    free(zero_ptr);
    zero_ptr = NULL;

    printf("All allocated memory has been freed.\n");
    return 0;
}
```

```
(base) saiabburi@Charishma-MacBook-Air OS_Session8 % clang -Wall -Wextra memory_allocation.c -o memory_allocation
(base) saiabburi@Charishma-MacBook-Air OS_Session8 % ./memory_allocation
Values using malloc: 10 20 30
Values after realloc: 10 20 30 40 50
Values using calloc: 0 0 0 0 0
All allocated memory has been freed.
(base) saiabburi@Charishma-MacBook-Air OS_Session8 % valgrind --version
zsh: command not found: valgrind
(base) saiabburi@Charishma-MacBook-Air OS_Session8 % leaks --atExit -- ./memory_allocation
memory_allocation(1511) MallocStackLogging: could not tag MSL-related memory as no_footprint, so those pages will be included in process footprint - (null)
memory_allocation(1511) MallocStackLogging: recording malloc and VM allocation stacks using lite mode
Values using malloc: 10 20 30
Values after realloc: 10 20 30 40 50
Values using calloc: 0 0 0 0 0
All allocated memory has been freed.
Process 1511 is not debuggable. Due to security restrictions, leaks can only show or save contents of readonly memory of restricted processes.

Process:         memory_allocation [1511]
Path:            /Users/USER/*/memory_allocation
Load Address:    0x104300000
Identifier:      memory_allocation
Version:         0
Code Type:       ARM64
Platform:        macOS
Parent Process:  leaks [1510]

Date/Time:       2026-10-13 11:16:56.664 +0530
Launch Time:     2026-10-13 11:16:56.594 +0530
OS Version:      macOS 13.4.1 (22F82)
Report Version:  7
Analysis Tool:   /usr/bin/leaks

Physical footprint:         2865K
Physical footprint (peak):  2865K
Idle exit:                  untracked
----

leaks Report Version: 4.0, multi-line stacks
Process 1511: 222 nodes malloced for 17 KB
Process 1511: 0 leaks for 0 total leaked bytes.
```

Task 2: Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
  UW PICO 5.09                                                File: copy_on_write.c


#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int data = 100;

    /* Ensure output appears in the correct order */
    setbuf(stdout, NULL);

    printf("Before fork: PID = %d, data = %d, address = %p\n",
           (int)getpid(), data, (void *)&data);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* Child process */
        printf("\nChild before modification: data = %d, address = %p\n",
               data, (void *)&data);

        data = 200;

        printf("Child after modification:  data = %d, address = %p\n",
               data, (void *)&data);
        _exit(0);
    } else {
        /* Parent process */
        if (waitpid(pid, NULL, 0) < 0) {
            perror("waitpid");
            return 1;
        }

        printf("\nParent after child modification: data = %d, address = %p\n",
               data, (void *)&data);
    }
```

```
        printf("Child after modification:  data = %d, address = %p\n",
               data, (void *)&data);
        _exit(0);
    } else {
        /* Parent process */
        if (waitpid(pid, NULL, 0) < 0) {
            perror("waitpid");
            return 1;
        }

        printf("\nParent after child modification: data = %d, address = %p\n",
               data, (void *)&data);
    }

    return 0;
}
```

```
(base) saiabburi@Charishma-MacBook-Air OS_Session8 % nano copy_on_write.c
(base) saiabburi@Charishma-MacBook-Air OS_Session8 % clang -Wall -Wextra copy_on_write.c -o copy_on_write
./copy_on_write
Before fork: PID = 1532, data = 100, address = 0x16b25f568

Child before modification: data = 100, address = 0x16b25f568
Child after modification:  data = 200, address = 0x16b25f568

Parent after child modification: data = 100, address = 0x16b25f568
(base) saiabburi@Charishma-MacBook-Air OS_Session8 % ./copy_on_write
Before fork: PID = 1534, data = 100, address = 0x16f917568

Child before modification: data = 100, address = 0x16f917568
Child after modification:  data = 200, address = 0x16f917568

Parent after child modification: data = 100, address = 0x16f917568
(base) saiabburi@Charishma-MacBook-Air OS_Session8 %
```